Variational Gibbs states on Tuna-17

How we prepared thermal states of 2-, 3- and 4-spin chains on simulators and on a real quantum computer: the physics, the circuit, the loss, the optimizer, the hardware, our scripts and every experiment we ran.

Taha Cakmak · Quantum Thermalization project · status: 22 October 2026, 13:15

How to read this. Sections 1–5 explain the method from scratch (what we prepare, with which circuit, which loss, which optimizer, and how we check the result). Section 6 explains how a real quantum computer is used in general. Sections 7–8 map our code and simulators. Section 9 lists every experiment we ran, with its numbers. Section 10 interprets them, Section 11 lists the next steps, and Section 12 is a glossary. All numbers come from the saved result files in the repository (branch `tfd-study`).

One-paragraph summary. We prepare the thermal (Gibbs) state of a transverse-field Ising chain of n spins by preparing a thermofield double state on 2n qubits with a short variational circuit, following Sagastizabal et al. (npj Quantum Information 7, 130). For n = 2 we reproduced the experiment on Tuna-17 (fidelity 0.90–0.96 for $\beta \leq 1$). For n = 3 and n = 4 we ran the identical plan on an ideal simulator, on Quantum Inspire's QX emulator, on two noise models fitted to our own hardware data, and on Tuna-17. The noiseless backends agree with each other and reach 0.86–0.96 (3+3); on Tuna-17 the 3+3 fidelity falls from 0.75 ($\beta = 0.5$) to 0.30 ($\beta = 5$). The loss is dominated by hardware noise, not by the circuit.

1 · What we prepare and why it is hard

Thermal states

A quantum system in contact with a heat bath at temperature T settles into its **Gibbs state**

$$\rho_\beta = e^{-\beta H} / Z,\ Z = \mathrm{Tr}\, e^{-\beta H},\ \beta = 1/T \text{ (inverse temperature)}.$$

Written in the eigenbasis of H (energies E_j, eigenstates $|j\rangle$), it is a weighted mixture $\rho_\beta = \Sigma_j\, e^{-\beta E_j}\, |j\rangle\langle j| / Z$. Two limits help intuition:

- **β → 0 (infinite temperature):** every eigenstate has equal weight; ρ is the maximally mixed state.
- **β → ∞ (zero temperature):** only the ground state survives; ρ becomes pure.

Preparing such states matters for simulating materials and chemistry at finite temperature. The difficulty: a quantum circuit acting on qubits that start in $|0\ldots0\rangle$ always produces a **pure** state, while a Gibbs state is **mixed**.

The trick: a thermofield double (TFD)

Take two copies of the system, A and B (each n qubits), and prepare the pure state

$$|\mathrm{TFD}(\beta)\rangle = \Sigma_j\, e^{-\beta E_j/2}\, |j\rangle_A\, |j\rangle_B / \sqrt{Z}.$$

If we now ignore B (mathematically: take the partial trace over B), what remains on A is exactly ρ_β. The mixedness of A comes from its entanglement with B. So: **preparing a pure state on 2n qubits gives a thermal state on n qubits.** At β = 0 the TFD is simply n Bell pairs $(|00\rangle+|11\rangle)/\sqrt{2}$, one between each A_i and B_i; this is why the circuit starts from Bell pairs.

Our model

System A is a transverse-field Ising chain with open ends and field g = 1:

$$H_A = \Sigma_i\, Z_i Z_{i+1} + g\, \Sigma_i\, X_i$$

For n = 2 this is $Z_1Z_2 + X_1 + X_2$ (the paper's model); for n = 3 there are two bonds, for n = 4 three. B has the same Hamiltonian. We write the experiments as **2+2, 3+3, 4+4** (A qubits + B qubits). Qubit order everywhere: $A_1\ldots A_n$ = q0…q(n−1), $B_1\ldots B_n$ = qn…q(2n−1).

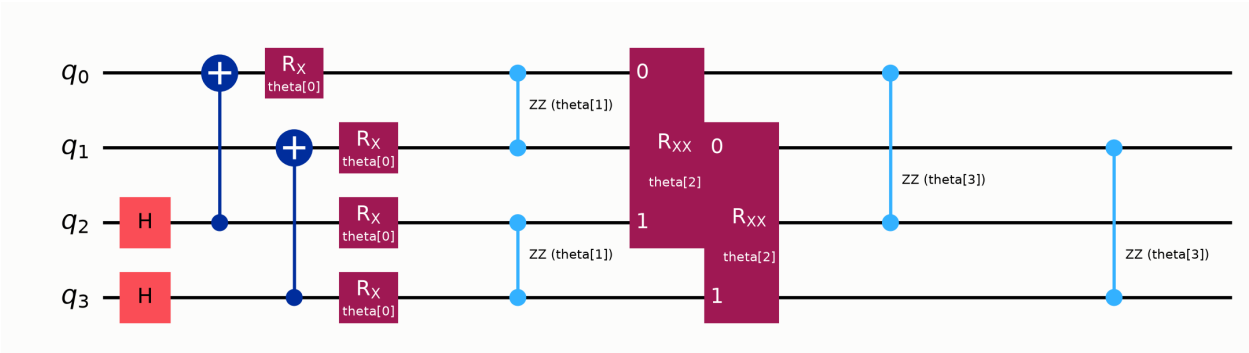
2 · The variational circuit (ansatz)

We do not know the circuit that makes $|\mathrm{TFD}(\beta)\rangle$ exactly, so we use a **parametrised circuit** and tune its angles until its output is close to the TFD. Our ansatz, the paper's, generalised to n spins and p layers:

Step	Gates	Acts on	Angle
0. Start	H on B_i, then CNOT $B_i \to A_i$	each rung A_i–B_i	none (Bell pairs = exact TFD at β = 0)
1. Field	$R_X(\gamma_1) = e^{-i\gamma_1 X/2}$	every qubit	γ_1
2. Chain	$R_{ZZ}(\gamma_2) = e^{-i\gamma_2 ZZ/2}$	bonds inside A and inside B	γ_2
3. Rungs	$R_{XX}(\alpha_1)$, then $R_{ZZ}(\alpha_2)$	each rung A_i–B_i	α_1, α_2

Steps 1–3 form one **layer**; they can be repeated p times. Each layer has only **4 angles** (γ_1, γ_2, α_1, α_2), shared by every gate of its kind, whatever n is. Why these gates: steps 1–2 mirror the

terms of H_A (X and ZZ), and step 3 couples A to B with the same operators that appear in H_{AB} (Section 3), which controls how entangled A and B are, and therefore how hot A looks.



The 2+2 ansatz with one layer (q0, q1 = A_1, A_2; q2, q3 = B_1, B_2). theta[0..3] = γ_1, γ_2, α_1, α_2. For 3+3 and 4+4 the same pattern has more chain bonds and rungs. Code: `tfd_study/model.py`, `ansatz(n, p)`; Omar's 3-spin circuit is identical (state overlap 1.0).

How good can this circuit be?

Even with perfect angles a short circuit cannot make every TFD exactly. Without noise, the best one-layer angles reach (fidelity of A to the Gibbs state):

	β = 0.5	**β = 1**	**β = 2**	**β = 5**
3+3, p = 1	0.993	0.972	0.952	0.962
4+4, p = 1	0.989	0.949	0.904	0.922

So one layer is already within a few percent of perfect. On hardware every two-qubit gate becomes CZ gates (the chip's native two-qubit gate). After compiling: **3+3, p = 1: 17 CZ; 3+3, p = 2: 31 CZ; 4+4, p = 1: 24 CZ.** Each CZ adds noise (Section 6), which is why we use p = 1.

3 · The loss function

On hardware we cannot read the state; we only get measurement counts. Measuring the fidelity directly needs full tomography (many circuits, Section 5), far too expensive inside an optimization loop. The paper therefore uses an **engineered cost** that needs only two kinds of measurement:

$$C_\varsigma(\beta) = \langle X_A + X_B \rangle + \varsigma \langle ZZ_A + ZZ_B \rangle - \beta^{-\varsigma} \langle H_{AB} \rangle$$

Term	**Meaning**	**3+3 example**
X_A	sum of X on all A qubits	$X_{A1} + X_{A2} + X_{A3}$
ZZ_A	sum of $Z_i Z_{i+1}$ over A's chain bonds	$Z_{A1}Z_{A2} + Z_{A2}Z_{A3}$
H_{AB}	coupling across rungs: Σ ($X_{Ai}X_{Bi} + Z_{Ai}Z_{Bi}$)	three XX and three ZZ rung terms
ς	tuning exponent (paper: 1.57)	we use ς = 1.57

Intuition. At high temperature (small β) the factor $\beta^{-\varsigma}$ is huge, so minimising C means maximising $\langle H_{AB} \rangle$: A and B must be strongly correlated, i.e. close to the Bell pairs. At low temperature the first two terms dominate and push A (and B) toward low energy. In between, the minimum of C is close to the TFD; ς sets how the two parts are balanced. C is a proxy, not the free energy itself, so even its exact minimum is slightly off the true TFD (this is part of the gap in the table of Section 2).

Choice of ς. Our noiseless scan (`run_noiseless_scan.py`, notebook 01) over ς and depth found: for one layer, ς ≈ 1.5 (2+2) and 1.57 (3+3, 4+4) is best, within 3% of the optimum; for two or more layers ς ≈ 1.3 is better. Omar's Ξ-scan suggested ς ≈ 2.05 for 3 spins (a different

criterion).

How the loss is measured

Every term is a product of only X's or only Z's. So **two circuits per parameter point** give all of C, for any n:

- **all-Z circuit**: measure every qubit directly; gives ZZ_A, ZZ_B and the $Z_{Ai}Z_{Bi}$ part of H_{AB}.
- **all-X circuit**: rotate every qubit with $R_Y(-\pi/2)$ first, then measure; gives X_A, X_B and the $X_{Ai}X_{Bi}$ part.

Each expectation value is a **parity** of the measured bits. For a shot giving bit string s with probability p(s): $\langle Z_iZ_i\rangle = \Sigma_s\, p(s)\,(-1)^{s_i \oplus s_j}$. Code: `model.loss_terms` and `model.loss`.

> **History.** For a while our code used a rescaled version, $L = \beta^C C/(1+\beta^C)$, which keeps numbers of order 1 at every β. Multiplying by a positive constant does not move the minimum, so it gives the same angles. Since 21 October every run uses the paper's unscaled C; we re-ran all simulations and the results were unchanged within shot noise.

4 · The optimizer

SPSA in one paragraph

SPSA (Simultaneous Perturbation Stochastic Approximation) is made for noisy cost functions. At step k it picks a random direction Δ (every angle ±1), measures the cost at $\theta + c_k\Delta$ and $\theta - c_k\Delta$, and moves

$$\theta \leftarrow \theta - a_k \cdot [C(\theta + c_k\Delta) - C(\theta - c_k\Delta)] / (2c_k) \cdot \Delta$$

So one step costs **2 cost evaluations = 4 circuits = 1 hardware job**, however many angles there are. The step sizes shrink slowly ($a_k \propto (k+1+A)^{-0.602}$, $c_k \propto (k+1)^{-0.101}$, Spall's standard schedule). Three practical additions (`gibbs_tfd_reproduction/optimizers.py`):

- **Gain calibration:** before starting, a few random pairs measure how steep the cost is, and a is set so the first step moves about 0.3 rad. This makes SPSA independent of the cost's scale.
- **Step cap:** no angle moves more than 0.3 rad in one step, so one noisy estimate cannot throw the angles far away.
- **Averaging:** the result is the mean of the last quarter of the iterates, which removes leftover jitter.

The protocol around SPSA

- **Kicks.** θ = 0 (Bell pairs) is a point where every gradient vanishes, so SPSA cannot leave it. Before the first β we evaluate 8 random small angle sets (σ = 0.4 rad) and start from the best.
- **Warm-start chain.** We go through β = 0.5 → 1 → 2 → 5, each starting from the previous β's answer (nearby temperatures have nearby optimal angles).
- **Budget.** 40 SPSA steps per β for 3+3 and 4+4 (20 for the 2+2 runs). Our simulated SPSA study (notebook 03) showed 40 steps come within 0.01–0.04 of the best reachable value.
- **Replicas.** The whole chain is repeated with different random seeds; the spread gives error bars.

Omar's branch uses SciPy's **COBYLA** instead (one evaluation per step, no special handling of shot noise). With identical settings his three emulator runs ended at fidelities 0.63, 0.89 and 0.97, which is why we kept SPSA with replicas.

5 · Checking the result: tomography and fidelity

Quantum state tomography

To see the state we reconstruct its density matrix. Any n-qubit ρ can be written as $\rho = (1/2^n)\ \Sigma_P\ \langle P\rangle\ P$, summing over the 4^n Pauli strings P (each qubit I, X, Y or Z). We measure every qubit in the X, Y or Z basis, in all 3^n combinations (27 settings for 3 qubits, 81 for 4), compute every ⟨P⟩ from the counts, and sum. Measurement noise can make the result slightly unphysical (small negative eigenvalues), so we project it onto the closest physical state.

We measure A_i and B_i in the same basis, so the same circuits give **both ρ_A and ρ_B**; both should equal the Gibbs state. Code: `model.tomography_rho`.

Readout correction

Qubits are sometimes read wrongly (0 as 1 or 1 as 0). We run two calibration circuits (all qubits in |0⟩; all in |1⟩), build each qubit's 2×2 confusion matrix, and apply its inverse to the measured probabilities. On a long run this is repeated every 30 minutes because readout drifts.

Fidelity

$$F(\rho, \sigma) = (\ \mathrm{Tr}\ \sqrt{(\sqrt{\rho}\ \sigma\ \sqrt{\rho})}\ )^2$$

F = 1 means identical states. We use this (qiskit's) definition everywhere; the paper quotes √F, which is larger (e.g. F = 0.81 is √F = 0.90). A useful floor: the maximally mixed state already has some fidelity with any Gibbs state (more at small β), so F does not drop to 0 even for a useless device.

Two kinds of measured state in every run

Name	Angles	What it tells us
Best angles (code: "reference")	computed classically before the run: exact noiseless optimum of the loss at each β	how good the device is when the angles are perfect; any loss is hardware noise
SPSA result (code: "replica")	found by SPSA on the backend itself, from noisy measurements	the actual variational experiment: optimizer and hardware together

The best-angle states are measured at the **start** and again at the **end** of every run; if the two agree, the device did not drift during the run.

Manhattan plots

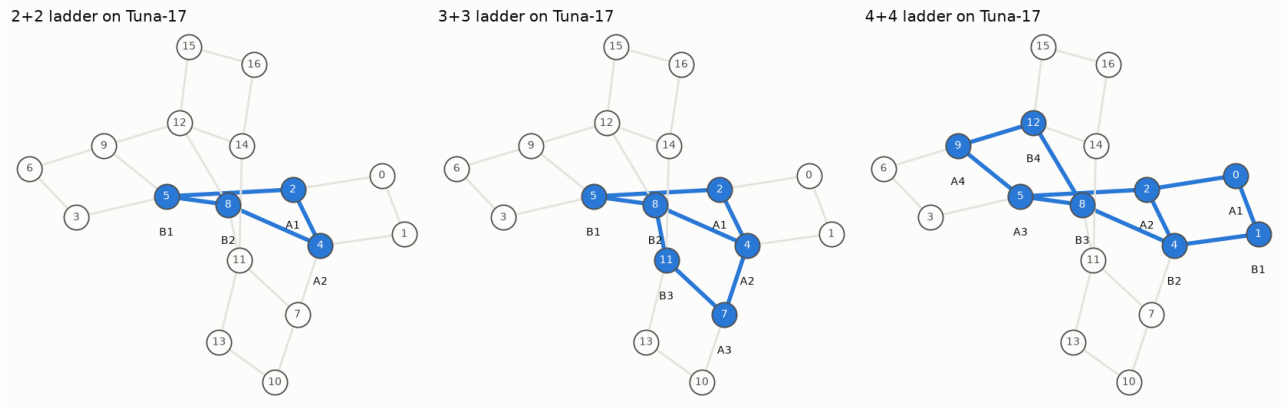
A Manhattan plot draws a density matrix as a grid of bars: bar (i, j) has height $|\rho_{ij}|$. The diagonal holds the populations of the basis states; off-diagonal bars are coherences. Our plots shade the bars light (small) to dark (large). The exact Gibbs state of our model is real, so any imaginary part in a measured state is an error; the plot titles state the largest $|\mathrm{Im}\ \rho_{ij}|$.

6 · How a real quantum computer is used

Everything in this section is general; the specifics are for Quantum Inspire (QI, QuTech, Delft) and its 17-qubit superconducting chip **Tuna-17**.

Concept	What it means	In our project
Access	an account and an API; jobs are sent over the internet	`qi login`, Python package `qiskit-quantuminspire`

Concept	What it means	In our project
Backend	a named device or simulator, with a status: IDLE, EXECUTING, CALIBRATING, BENCHMARKING, OFFLINE	Tuna-17 (hardware), QX emulator (QI's simulator); Tuna-9 and Tuna-5 are offline this week
Native gates	the few operations the chip physically does; everything else is compiled into them	rx, ry, rz, cz. rz is "virtual" (free, done in software); cz is the noisy part
Connectivity	a CZ only works between two qubits joined by a coupler; otherwise the compiler inserts SWAPs (3 CZ each)	we place the 2×n ladder on qubits whose bonds and rungs are all couplers: no SWAPs
Measurement	a qubit is read only as 0 or 1 (Z basis); other bases need a rotation first	$R_Y(-\pi/2)$ for X, $R_X(\pi/2)$ for Y
Shots	each circuit is repeated many times; an expectation value has statistical error ≈ 1/√shots	4096 shots → about ±0.016 per expectation value
Jobs and queue	circuits are sent in jobs and wait in a queue shared with other users	≤ 5 circuits per job, ≤ 5 jobs queued per user; 33 jobs/h on a busy day, much faster at night
Errors	CZ errors (~2%), decay and dephasing (T_1, T_2), readout errors, coherent (systematic) phase errors, drift over hours	readout: worst qubit 6–10% wrong; see Section 10
Outages	devices go offline (calibration, maintenance), networks drop	retries, saved state after every job, automatic resume (a supervisor script)



Tuna-17's couplers (grey) and the ladders we use (blue): 2+2 on qubits 2, 4, 5, 8; 3+3 on 2, 4, 7, 5, 8, 11; 4+4 on 0, 2, 5, 9, 1, 4, 8, 12. Every chain bond and rung is a physical coupler.

Compiling (transpiling)

Before sending, qiskit's transpiler maps logical qubits to the chosen physical qubits and rewrites the gates for the backend. One lesson from our 3+3 run: we compiled against QI's backend description, which also accepts cx and cp; the circuits we sent contained 9 cx + 7 cp, which QI then turns into CZs (a general cp needs two), about 23 CZ. Compiling to native CZ ourselves gives 17.

Anatomy of one of our hardware runs

`python -m tfd_study.run_hardware --config tfd_study/hardware_plan.json` (3+3):

- Reads the plan: n, p, ς, β list, qubits, replicas, SPSA steps, shots.
- Computes the best angles for each β on the laptop (for the best-angle measurements).
- Builds a to-do list of **tasks**: readout calibration (2 circuits), kicks (16 circuits), SPSA steps (4 circuits each), tomographies (27 or 81 circuits). Tasks are cut into jobs of ≤ 5 circuits.

- A **scheduler loop** keeps 4 jobs in QI's queue, collects finished ones, saves the raw counts, and when a task is complete analyses it: a calibration updates the readout correction, an SPSA step computes the loss and asks SPSA for the next point, a tomography reconstructs ρ_A, ρ_B and their fidelity.
- Replicas run interleaved so the queue never idles. Calibration is refreshed every 30 min.
- Everything is saved after every job, so the run can be resumed after any interruption.

A 3+3 run is about 470 jobs; last night it took 5.7 h. A 4+4 run is about 405 jobs.

7 · Our software, file by file

Early 2+2 code: `gibbs_tfd_reproduction/`

File	What it does
tfd_circuit.py, cost_function.py, ground_truth.py	2+2 circuit, the paper's cost, exact Gibbs/TFD states
optimizers.py	SPSA with gain calibration, step cap and resumable state (used everywhere)
run_hardware.py	first hardware runner: one chain, one job at a time (the "night-1" run)
run_hardware_parallel.py	parallel scheduler: tasks, ≤ 5-circuit jobs, queue of 4–5, calibration refresh, checkpoints (the main 2+2 run; reused by tfd_study)
analyze_parallel_run.py	tables and plots of a 2+2 run
*_3spin.py, extension_experiment.py, plot_3spin.py	Omar's 3-spin version (same circuit and cost), COBYLA runner for QI, his Manhattan plots and Ξ(ς) validation

The n+n study: `tfd_study/`

File	What it does
model.py	physics for any n, p: H_A, Gibbs state, ansatz, measured circuits, loss, tomography, readout correction, exact references
devices.py, tuna17.json	Tuna-17 coupling map, ladder placements, compiling to native gates, the two noise models, simulators
run_hardware.py	**the experiment runner**: same plan on hardware, QX emulator or a local simulator (--simulate ideal | uniform | coherent); --self-test, --dry-run, --resume
protocol.py	the SPSA chain inside simulations (with hardware cost bookkeeping)
run_noiseless_scan.py	study 1: best ς and depth p without noise (notebook 01)
fit_noise.py	fits the Tuna-17 noise models to our 2+2 hardware tomographies (notebook 02)
run_ceilings.py	best reachable fidelity at the hardware β's, with and without noise
run_spsa_study.py	study 2: SPSA with a hardware-sized budget on the noise model (notebook 03)
analyze_run.py	re-simulates a hardware run's exact compiled circuits; can refit noise on a run
compare_sim_runs.py, plot_density.py, plot_manhattan.py	comparison tables and density/Manhattan plots
hardware_plan.json, hardware_plan_4.json	the 3+3 and 4+4 Tuna-17 plans

File	What it does
qx_plan.json, qx_plan_4.json, sim_plan_4_r3.json	QX emulator plans; 4+4 with 3 replicas for simulators
notebooks/01–04	only load results and plot: design, noise model, SPSA study, hardware plan

Commands

```
python -m tfd_study.run_hardware --config tfd_study/hardware_plan.json --self-test
python -m tfd_study.run_hardware --config tfd_study/hardware_plan.json --simulate coherent
python -m tfd_study.run_hardware --config tfd_study/qx_plan.json
python -m tfd_study.run_hardware --config tfd_study/hardware_plan.json # Tuna-17
python -m tfd_study.run_hardware --resume results/tfd_n3p1_<tag>
python -m tfd_study.compare_sim_runs
python -m tfd_study.plot_manhattan results/tfd_n3p1_<tag>
```

What a run saves (results/<run>/)

File	Content
raw_jobs.jsonl	raw counts of every job, with its task (β, replica, angles) and time
compiled.jsonl	every circuit exactly as compiled and sent (OpenQASM 3)
state.json	plan, best angles, full SPSA history, every reconstructed ρ_A, ρ_B with fidelity
analysis.txt	the summary table

Every plot in this report can be regenerated from these files.

8 · Simulators and noise models

Backend	What it is	Shots
Ideal simulator	qiskit Aer, perfect gates; only shot noise	4096
QX emulator	Quantum Inspire's own simulator, run through the same QI interface as Tuna-17 (all-to-all, 10 qubits)	2048 (its maximum)
Noise model: uniform	Aer with one error model for the whole chip, fitted to our 2+2 Tuna-17 tomographies: per CZ depolarizing 0.023 plus amplitude damping 0.027 per qubit; per 1-qubit gate 0.0012; readout 3.7% / 3.1%	4096
Noise model: coherent	as above but per coupler, with a systematic phase error after each CZ; fits the 2+2 data better (held-out error 0.030 vs 0.038); unmeasured couplers get random phases of the fitted size	4096

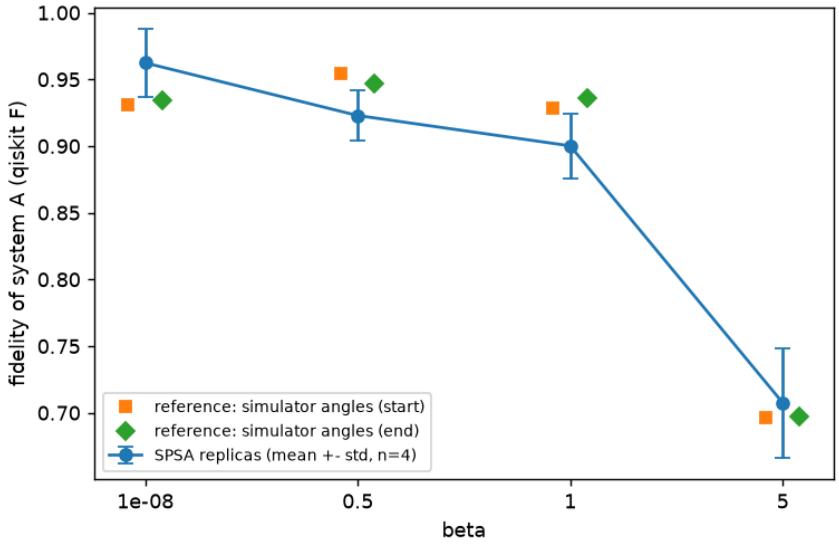
Both noise models were fitted on **4 qubits** (2, 4, 5, 8). Marios proposed a different, spec-sheet style simulator (qi-utilities' NoisySimulator: per-qubit T_1/T_2, RB and IRB errors, readout), with decoy devices for Tuna-9 (3+3) and an 8-qubit ladder (4+4); that is not built yet.

9 · Everything we ran

2+2: reproducing the paper on Tuna-17

Run (folder)	When	Setup	Purpose
hardware_20261019_001940 ("night-1")	18–19 Oct	one chain, β ≈ 0, 0.5, 1, 5; 20 SPSA steps; 8192 shots; one job at a time	first full hardware run
parallel_20261019_175824 (main)	19–20 Oct, ~18 h, 603 jobs	4 replicas, 20 SPSA steps per β, 8192 shots (cost) / 4096 (tomography), full 16-state readout calibration, qubits 2, 4, 5, 8	the reproduction; also the data our noise models are fitted to

β	SPSA F_A (4 replicas)	best angles, start / end	best possible (no noise)
≈ 0 (10^{-8})	0.962 ± 0.025	0.931 / 0.935	1.000
0.5	0.923 ± 0.019	0.954 / 0.947	0.997
1	0.900 ± 0.024	0.929 / 0.936	0.993
5	0.708 ± 0.041	0.696 / 0.697	0.987



2+2 on Tuna-17. Blue: SPSA replicas (mean ± std). Orange/green: best angles measured at start/end of the run. Start and end agree, so the device was stable for 18 h.

Reading: above 0.9 up to β = 1; at β = 5 even perfect angles give only 0.70, so low temperature is limited by the hardware, not by SPSA.

Design studies for 3+3 and 4+4 (simulation only)

- **Noiseless scan**: one layer reaches 0.90–0.99; ς = 1.57 is right for p = 1.
- **Noise model** fitted to the 2+2 data (two variants, Section 8).
- **SPSA study**: with the noise model, mean device fidelity for 3+3 is 0.77 / 0.67 / 0.59 for p = 1 / 2 / 3, so **one layer wins** at every size; 40 steps per β are enough.
- Result: the hardware plans (hardware_plan.json, hardware_plan_4.json).

3+3 and 4+4: the same plan on five backends

Setup for every backend: p = 1, ς = 1.57, β = 0.5, 1, 2, 5, 40 SPSA steps per β, 8 kicks, 4096 shots (QX 2048). 3+3: 2 replicas; 4+4: 3 replicas on the local simulators, 1 on QX and Tuna-17. Tuna-17 runs: 3+3 on 21–22 Oct overnight (457 jobs, 5.7 h, no failed job); 4+4 on 22 Oct 05:36–13:03 (two short device outages, both recovered automatically).

3+3, fidelity F_A after SPSA (mean over replicas)

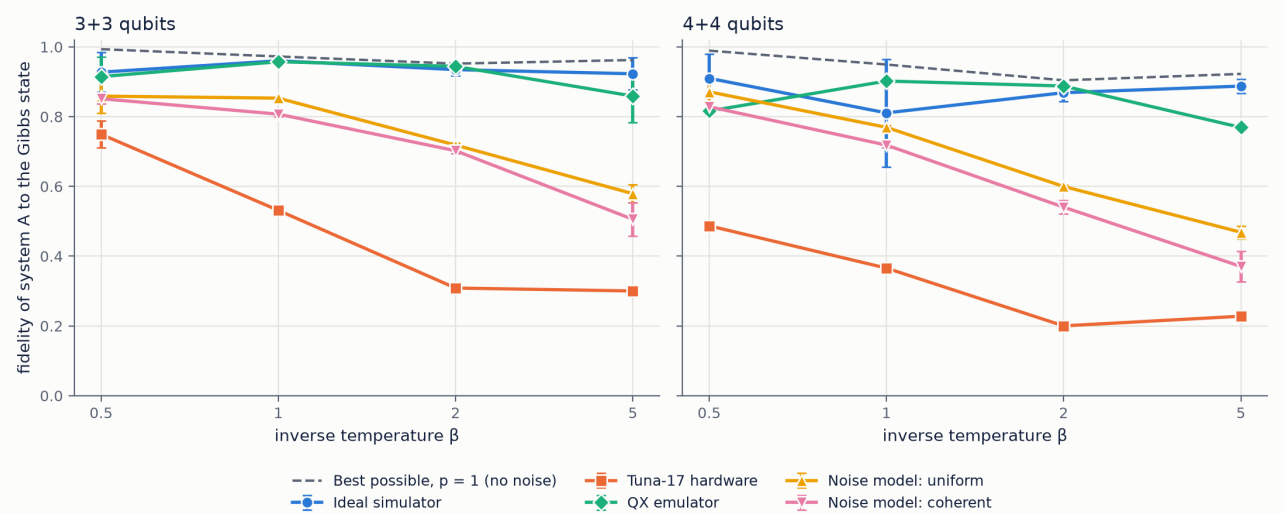
Backend	β = 0.5	β = 1	β = 2	β = 5
Best possible, p = 1	0.993	0.972	0.952	0.962
Ideal simulator	0.928	0.959	0.938	0.920
QX emulator	0.915	0.957	0.944	0.859
Noise model: uniform	0.862	0.851	0.732	0.573
Noise model: coherent	0.848	0.785	0.684	0.496

Backend	β = 0.5	β = 1	β = 2	β = 5
Tuna-17, SPSA	0.749	0.531	0.309	0.300
Tuna-17, best angles (start)	0.761	0.596	0.471	0.361

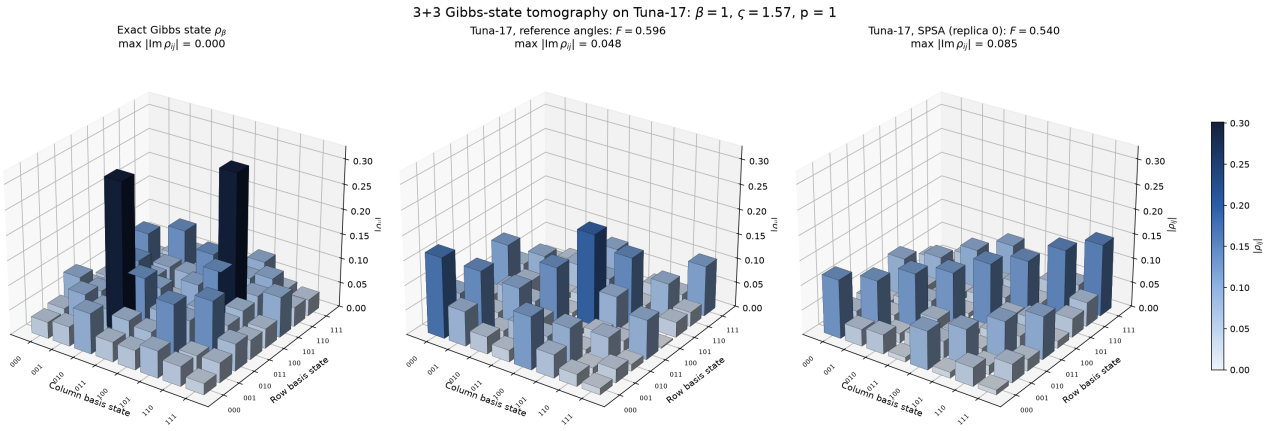
4+4, fidelity F_A

Backend	β = 0.5	β = 1	β = 2	β = 5
Best possible, p = 1	0.989	0.949	0.904	0.922
Ideal simulator (3 replicas)	0.910	0.810	0.869	0.890
QX emulator (1 replica)	0.816	0.902	0.888	0.769
Noise model: uniform	0.867	0.769	0.600	0.463
Noise model: coherent	0.822	0.710	0.536	0.374
Tuna-17, SPSA	0.487	0.366	0.200	0.228
Tuna-17, best angles (start)	0.550	0.383	0.358	0.122
Tuna-17, best angles (end)	0.645	0.477	0.446	0.135

The best-angle states were about 0.1 better at the end of the 4+4 run than at the start: the device drifted during those 7 hours, so the 4+4 hardware numbers carry roughly that much uncertainty. (For 3+3, start and end agreed within about ±0.06.)



Fidelity after SPSA versus β on every backend (error bars: spread over replicas). Dashed: best possible with one layer and no noise.



3+3 at β = 1. Left: exact Gibbs state (two dominant elements at 010 and 101). Middle: best angles on Tuna-17 (F = 0.596). Right: SPSA angles on Tuna-17 (F = 0.540). The peaks flatten toward a mixed state, and imaginary parts up to 0.05–0.09 appear where the exact state is real.

Omar's 3+3 work

Same circuit and Hamiltonian (checked numerically), run only on the QX emulator with COBYLA and ς = 2.05; his branch is merged into tfd-study. His results vary strongly between identical runs (0.63–0.97), consistent with COBYLA's sensitivity to shot noise.

10 · What the results mean

- **The software is right.** The ideal simulator and QX (a different simulator, reached through QI like the hardware) agree for 3+3 and 4+4.
- **Noise, not the circuit, limits us.** 3+3 at β = 1: perfect angles on a perfect device give 0.972; the same angles on Tuna-17 give 0.596. About 3% is lost to the one-layer circuit and 37% to noise.
- **More layers would not help.** They can win back at most the 3% and add 14 CZ per layer; with our noise model 3+3 drops from 0.77 (p = 1) to 0.67 (p = 2).
- **Tuna-17 was noisier than our models predicted** (3+3 at β = 1: 0.60 measured vs 0.83–0.86 modelled). Likely reasons: the models were fitted on 4 qubits and this run used 6 qubits and 7 couplers; readout was weak (worst qubit 90–94% correct over 9 calibrations); the circuits we sent had about 23 CZ instead of 17; the device had just returned from fridge maintenance.
- **Coherent phase errors are visible:** imaginary parts where the target is real.
- **SPSA behaves.** Hardware replicas agree within 0.001–0.04. Without noise SPSA occasionally stalls (one 4+4 ideal replica: 0.59 at β = 1), which is why we use replicas.

11 · Next steps

- A/B test on Tuna-17: native-CZ compiling (17 CZ) versus the current circuits, best-angle states only (~50 jobs); rerun 3+3 if it helps.
- Undo coherent phase errors with free virtual-Z rotations, fitted from the best-angle tomographies.
- Choose better qubits with quick best-angle tests on other ladder placements.
- Build Marios's decoy simulators (Tuna-9 for 3+3, 8-qubit ladder for 4+4).
- Refit the noise model on the new 3+3/4+4 hardware data (analyze_run.py).

12 · Glossary

Term	Meaning
β	inverse temperature 1/T; small β = hot, large β = cold
Gibbs state	thermal equilibrium state $e^{-\beta H}/Z$
TFD	thermofield double: pure state on A+B whose A part is the Gibbs state
Partial trace	the operation “ignore system B”; gives the state of A alone
Ansatz	parametrised circuit whose angles are optimized
Layer (p)	one repetition of the ansatz’s gate pattern; we use p = 1
ς	exponent in the engineered loss (1.57)
SPSA	optimizer that estimates the gradient from two measurements per step
Replica	an independent repeat of the whole optimization with another random seed
Kick	random starting angles to leave the stuck point θ = 0
Shot	one execution and measurement of a circuit
Job	a batch of ≤ 5 circuits sent to Quantum Inspire
Native gate	an operation the chip does physically (rx, ry, rz, cz on Tuna-17)
Transpile / compile	rewrite a circuit into native gates on chosen physical qubits
Layout / ladder	which physical qubits play $A_1 \ldots B_n$; chosen so every needed pair is coupled
Tomography	reconstructing ρ from measurements in many bases
Readout correction	undoing measurement errors with calibration circuits
Fidelity F	overlap of two states, 1 = identical (we use qiskit’s squared definition)
Best angles (“reference”)	classically computed optimal angles, measured on the device to isolate noise
Coherent error	a systematic, repeatable unwanted rotation (e.g. a phase), unlike random noise
QX emulator	Quantum Inspire’s simulator, used through the same interface as the hardware